PiLLM: Resource-efficient LLM Inference Using Workload Prediction

Yunqian Fan*
fanyq2022@shanghaitech.edu.cn
School of Information Science and Technology, ShanghaiTech University, Shanghai, China

Shihao Bai
baishihao@sensetime.com
SenseTime Research

Ruihao Gong†
gongruihao@buaa.edu.cn
Beihang University, Beijing, China

Zaijun Wang
wangzaijun@sensetime.com
SenseTime Research

Rui Fan†
fanrui@shanghaitech.edu.cn
School of Information Science and Technology, ShanghaiTech University, Shanghai, China

Abstract

LLM inference demands substantial GPU resources, but its highly variable workload characteristics create efficiency challenges at both inter-GPU and intra-GPU levels. To meet Service Level Objectives (SLOs), existing systems typically overprovision resources in two ways: allocating excess GPUs to handle peak loads and reserving excessive memory per request to prevent out-of-memory during token generation. We introduce PiLLM (Predictable inference for LLMs), a system that addresses these inefficiencies through accurate workload prediction and dynamic resource allocation.

PiLLM's core lies in the statistical prediction algorithm that accurately models computational and memory requirements by analyzing input/output sequence lengths distribution and request patterns. This enables two complementary optimizations: (1) an elastic inter-GPU dispatch mechanism that dynamically scales GPUs based on workload forecasts, and (2) an intra-GPU scheduler that improves utilization but reduces eviction through precise memory allocation.

We implement PiLLM on top of the disaggregated prefill/decode paradigm for fine-grained GPU saving. Extensive evaluation on production workloads with load fluctuations demonstrates that PiLLM significantly improves resource efficiency without compromising SLOs. Our inter-GPU dispatch reduces average usage by 1.6- 3.1x while maintaining SLO compliance, and our intra-GPU scheduler achieves throughput improvements with negligible eviction rates (<0.1%).

CCS Concepts: • **Computer systems organization → Cloud computing**; • **Computing methodologies → Distributed artificial intelligence**; **Natural language processing**; • **Theory of computation → Scheduling algorithms**.

Keywords: Large Model Inference, GPU Management, Efficient Computation, Load Prediction, Quality of Service

*Major part of this work was done during an internship at SenseTime.
†Corresponding authors

1 Introduction

Large Language Models (LLMs) demonstrate powerful generalization capabilities and can perform numerous practical tasks via inference. However, LLM inference systems differ fundamentally from traditional ML inference workloads (e.g., image classification) which only run once. It involves repeated invocations during auto-regressive generation, which is especially evident in current compound applications such as Chain of Thought (CoT) [17], reasoning frameworks [2, 13], and AI agents [7]. These applications require real-time responsiveness, significantly increasing both frequency and computational demands of online inference, driving organizations to deploy substantial GPU clusters for LLM serving.

However, average GPU utilization during LLM online inference remains relatively low, resulting in substantial waste of computational resources. This inefficiency stems from highly variable computation loads and strict Service Level Objectives (SLOs). Figure 1 illustrates this problem using a one-day trace from a production LLM inference system. While request frequency follows a predictable diurnal pattern, computational demand exhibits dramatic unpredictability with minimal correlation to request volume. This mismatch arises from two critical characteristics of LLM inference workloads: (1) both inputs and outputs have highly variable lengths, creating unpredictable memory requirements; and (2) the attention mechanism has quadratic complexity to sequence length, further amplifying the variability [15]. It creates overprovision challenges at both intra-GPU and inter-GPU levels. At the inter-GPU level, providers must maintain surplus GPUs to handle load spikes from long or complex requests. At the intra-GPU level, systems must reserve excessive memory for key-value cache(KV cache) generated by LLMs to avoid out-of-memory issues during token generation.

Existing solutions fall short on both inter-GPU and intra-GPU management. At the inter-GPU level, auto-scaling for

EUROSYS '26, Edinburgh, Scotland Uk

ACM ISBN 979-8-4007-2212-7/26/04
https://doi.org/10.1145/3767295.3769393

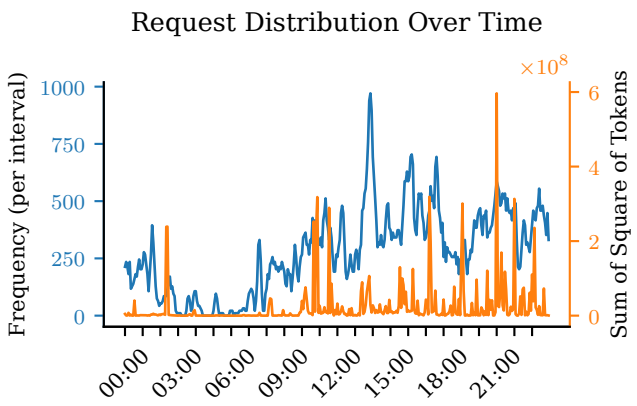

Figure 1. Disparity between request frequency and computational demand in production LLM inference.

LLMs uses GPU utilization metrics [1] that fail to detect load spikes from long requests, which maintain steady utilization while degrading service quality. At the intra-GPU level, current schedulers face an efficiency-reliability dilemma: aggressive memory allocation improves utilization but risks out-of-memory evictions [11], while conservative approaches ensure stability but waste resources [9].

One shared limitation of these approaches is the inability to predict real-time computational load and request memory requirement accurately. With reliable prediction, both inter- and intra-GPU scheduling could be optimized as a semi-deterministic problem. While recent transformer-based approaches can predict output lengths, they struggle with input length variability and require additional GPU resources to run the prediction models, which is impractical for real-time scheduling and detracts from the resources available for the inference process and potentially degrades overall system QoS. Also, the variation between LLM requests makes predicting individual request characteristics infeasible.

While individual request lengths are highly variable, the statistical properties of large batches of requests become increasingly predictable as the request count grows. As more requests are aggregated, the average behavior converges toward a stable distribution (Law of Large Numbers), allowing us to make reliable predictions about input/output length patterns despite individual variations. LLM inference system always operates on a batch of requests, based on the batched metrics like total Flops and memory, etc. Thus, the batched prediction aligns with LLM inference. Batched prediction enjoys properties such as reduced error bound, along with batch size and calculation efficiency. This motivates us to develop a lightweight statistical batched resource predictor, towards FLOPs and memory of realtime workloads, in which time window aggregation naturally batches requests. Unlike transformer-based predictors, our approach introduces minimal computational overhead while providing sufficient accuracy for scheduling decisions.

Using this predictor, we propose PiLLM, a framework targeting at reducing resource waste while maintaining service quality via prediction. Both inter-GPU and intra-GPU scheduling are specially designed around batched workload prediction to address the identified challenges. For inter-GPU scheduling, we calculate total computational requirements (FLOPs) based on predicted average request length and forecasted request count, then derive the optimal GPU count as a function of execution time requirements and computational load. This approach replaces utilization metrics in auto-scaling with a measure of actual workload, enabling precise resource allocation while maintaining SLOs. We further enhance its robustness with a fallback mechanism to launch new instances immediately for unexpected load spikes. For intra-GPU scheduling, our workload predictor provides accurate estimates of future KV cache requirements, allowing the scheduler to achieve both high memory utilization and low eviction rates simultaneously, overcoming the traditional trade-off between these competing objectives.

To maximize resource efficiency, we implement PiLLM on a disaggregated prefill/decode paradigm where these two computation phases can be scheduled independently. This architectural choice enables more fine-grained inter-GPU resource management because the compute-intensive prefill phase and the memory-access-intensive decode phase have distinct resource requirements.

In summary, our key contributions are as follows:
(1) We identify key inefficiencies in LLM inference systems as unpredictability and develop a lightweight statistical prediction algorithm that models request length distributions.
(2) We design an elastic inter-GPU dispatch mechanism that dynamically scales resources based on predicted computational requirements, reducing GPU usage by 1.6-3.1 times while maintaining SLO compliance.
(3) We develop an intelligent intra-GPU scheduler that optimizes memory allocation based on predicted KV cache requirements, achieving 99.5% of state-of-the-art GPU utilization while reducing eviction rates to less than 0.5%.
(4) We call the prediction-aware system PiLLM and implement it on a disaggregated prefill/decode architecture for fine-grained inter-GPU management. It is evaluated on real-world workloads with natural load fluctuations, demonstrating significant improvements across inter- and intra-GPU levels.

2 Related Work

2.1 Predictable ML Inference Systems

DNN Inference Predictability. Clockwork [6] pioneered predictable DNN serving by revealing that its inference is highly deterministic, thus redesigns a multi-level tight scheduling while maintaining consistent quality. Shepherd [19]

extends this approach to request intervals by grouping them to predict arrival patterns, thereby reducing variability and improving resource allocation efficiency. Both systems rely on the fundamental facts that DNN computational loads exhibit high predictability and even the arriving patterns show predictability when grouping, which inspired our exploration of statistical methods to predict LLM loads in a grouping manner against their inherently higher variability.

Uncertainty in LLM Systems. Unlike traditional DNNs, which have fixed computational graphs with predictable memory and processing requirements, LLMs present unique scheduling challenges due to their transformer architecture and autoregressive generation pattern. While DNNs execute the same operations regardless of input content, LLM computation scales quadratically with sequence length and requires maintaining growing key-value caches during generation. This fundamental difference means scheduling techniques optimized for DNNs' predictable workloads cannot be directly applied to LLMs. Existing approaches attempting to address LLM-specific challenges include transformer-based output prediction [21] and distribution sampling [4], but these methods often introduce significant computational overhead or struggle with the combined variability of both input processing and output generation.

LLMSched [23] addresses uncertainty on the stage change and completion time in multi-stage LLM applications by modeling workflows as DAGs composed of different stage types, then calibrating the probability of different stages as a Bayesian Network. This approach quantifies stage switching and completion time uncertainty through entropy, thus enabling priority scheduling of stages that most effectively reduce overall uncertainty. LLMSched demonstrates that LLM computation loads can also be probabilistically modeled, which motivates our statistical prediction approach rather than relying on more computationally expensive neural methods.

2.2 Resource Management and LLM Scheduling

Auto Scaling for Cloud Service. KServe [10] provides inference serving capabilities with auto-scaling that performs well for traditional ML inference frameworks like PyTorch and TensorFlow. However, KServe-based services rely on generic utilization metrics that fail to capture the spike characteristics inherent to LLM workloads, thus causing either resource underprovisioning during spikes or wasteful overprovisioning during normal operation [1]. These spikes occur frequently in LLM applications due to variable input/output lengths, while traditional ML inference rarely encounters such variation because computational and memory loads remain almost fixed.

LLM Execution Paradigm. Traditional LLM serving follows a monolithic approach where prefill and decode phases execute on the same hardware, treating the whole prefill and each decode step as atomic operations while allowing prefill to preempt decode steps [18]. As LLM context windows grow, this pattern leads to severe interruption of the decode phase due to the mismatch in execution time between prefill and decode steps, thus reducing the quality of decode. Chunked prefill strategies like SplitFuse [8] segment long prefill computations to reduce latency impact on decode operations, thereby improving responsiveness without fully separating the phases. In contrast, disaggregated prefill/decode completely separates these phases across dedicated resources, thus allowing them to run independently and reducing the interruption cost to mere state transfer time [22].

Batching Strategy. Current research on batching primarily addresses KV Cache management in GPU memory. Existing strategies lack accurate predictions of input/output lengths, forcing schedulers to either over-estimate or under-estimate memory needs [9, 11]. This leads to suboptimal tradeoffs between GPU utilization and eviction rates. While some methods attempt to predict output lengths using transformers [21] or distribution sampling [4], their high computational complexity and request-wise nature make them impractical for modern LLM inference systems handling numerous concurrent requests.

3 Preliminary

3.1 LLM Inference Process

LLM inference consists of prefill and decode phases, with computational characteristics fundamentally different from traditional DNN inference.

In this work, r_i denotes the i-th inference request, while batch $\mathcal{B}$ represents requests in processing and $\mathcal{Q}$ the pending queue. Each request r_i has an input sequence length $l_{p,i}$, current output length $l_{d,i}$, maximum allowed output length $l_{m,i}$, and the actual output length $l'_{d,i}$. The subscripts p and d indicate whether the length is used by the prefill (input processing) or decode (token generation) phase.

The prefill phase processes the entire input sequence at once, with quadratic computational complexity $O(l_{p,i}^2)$ relative to input length. Due to variable input lengths, prefill computation can vary by orders of magnitude between requests, unlike DNNs with fixed-dimension inputs and thus fixed computation amount.

The decode phase generates tokens sequentially, with each generating step having linear complexity $O(l_{p,i} + l_{d,i})$ relative to the current sequence length. This phase is memory-intensive. While each step has linear complexity, the entire decode process still exhibits quadratic complexity across all generated tokens.

LLM has a highly variable computation amount and an increasing memory need: prefill is compute-bound with higher intensity, while decode is memory-bound with lower per-step intensity. Both contribute to growing memory usage as

the KV cache expands, requiring careful management. We manage GPUs in model instance granularity, where each instance contains the complete model parameters needed to perform inference. n_p and n_d indicate the instances allocated for prefill and decode phases, respectively. The KV cache memory consumption for a sequence of l tokens is $c_{\text{kv}} \cdot l$.

3.2 Uncertainty Modeling

While request arrival patterns also exhibit uncertainty, existing techniques can address this. The unique challenges in LLM inferences are the varied input/output lengths, as discussed in section 2, with inputs ranging from simple queries to complex documents and outputs varying based on prompt content, sampling parameters, and stopping conditions. Both follow long-tail distributions that significantly complicate resource planning. Our work focuses on the input/output length distributions. We model them as input length distribution ($l_p \sim \Gamma_p$) and output length distribution ($l_d \sim \Gamma_d$).

3.3 Intra- and Inter-GPU Scheduling

LLM inference systems operate with two complementary scheduling objectives: minimizing total GPU count through inter-GPU scheduling and maximizing per-GPU utilization through intra-GPU scheduling.

Inter-GPU scheduling determines the number of model instances (n_p, n_d) to allocate for the current and next period load. This requires predicting overall computational requirements based on arrival patterns and load distributions. Current approaches like KServe rely on hardware metrics rather than LLM-specific characteristics, thus suffer lag, leading to either underprovisioning during spikes or wasteful overprovisioning during normal operation.

Intra-GPU scheduling manages KV cache memory allocations and request batching within GPUs. Continuous batching systems must decide which requests to batch or evict based on memory constraints and estimated resource needs ($c_{\text{kv}} \cdot (l_{p,i} + l_{d,i})$ v.s. $e(l_{p,i}, l_{d,i})$), which relies on per-request estimation. The estimation functions are designed to optimize for different objectives, such as memory utilization and request eviction[11, 12, 20]. However, the absence of accurate per-request predictions for input/output lengths compels existing systems to adopt suboptimal trade-offs. We found that statistical modeling of length distributions is beneficial, as it enables simpler yet effective batched predictions that achieve greater precision and resource utilization. Taking decode for instance, a batched average output length estimation is:

$$\hat{l}_{d,\mathcal{B}} = \mu_d + \frac{\sigma_d}{\sqrt{|\mathcal{B}|}} \cdot \Phi^{-1}(1-\epsilon), \tag{1}$$

where Φ^{-1} is the inverse of the standard normal cumulative distribution function. We will illustrate the parameter calibration in section 4.

4 Methodology

In this section, we present our approach to uncertainty challenges in LLM inference. We first introduce our statistical prediction method for input/output lengths, which performs best when applied to batches of requests rather than individual requests. Building on this foundation, we then redesign intra-GPU scheduling from request-level to batch-level memory management, enabling accurate KV cache estimation that simultaneously optimizes resource utilization and minimizes eviction rates. Finally, we develop a new inter-GPU management approach that dynamically allocates computational resources based on predicted FLOPs requirements and SLOs, with a robust fallback mechanism to handle unexpected load spikes.

4.1 Statistical Prediction for Input/Output Lengths

LLM serving systems struggle with uncertainty in both input and output lengths. Input lengths are known only after the requests arrive, and their distribution follows a heavy tail. Output lengths are also challenging as they remain unknown until generation completes. Both impact resource requirements significantly: input length affects prefill computation quadratically, while output length determines KV cache memory growth.

We observe that while individual request length prediction is difficult, the statistical properties of request batches are more predictable. The distributions of several scenarios are shown in Figure 2, where the distributions are stable and skewed to the left. We are motivated to model the length distributions Γ_p and Γ_d and leverage the Central Limit Theorem to improve prediction accuracy. However, in practice, we do not need a full set of data but only require a sliding window to update the statistics, which may contain hundreds of requests.

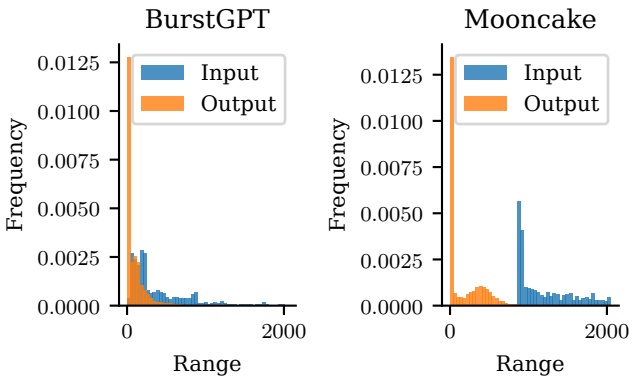

Figure 2. Input/Output Length Distributions For BurstGPT and MoonCake

For a batch of requests $\mathcal{B}$, the average output length $\bar{l}_{d,\mathcal{B}}$ is represented as $\bar{l}_{d,\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{r_i \in \mathcal{B}} l'_{d,i}$. Assuming the request's

actual output length $l'_{d,i}$ follows the distribution Γ_d with mean μ_d and variance σ_d^2, according to the Central Limit Theorem, the average output length $\bar{l}_{d,\mathcal{B}}$ will converge to a normal distribution: $\bar{l}_{d,\mathcal{B}} \sim \mathcal{N}(\mu_d, \frac{\sigma_d^2}{|\mathcal{B}|})$.

This means that as the batch size $|\mathcal{B}|$ increases, the variance of the average length decreases proportionally, making our prediction more accurate for larger batches. We can establish confidence intervals for our predictions, allowing the system to make informed trade-offs between resource efficiency and risk of memory exhaustion.

First, to estimate μ_d and σ_d^2, we maintain historical statistics of request lengths by scenarios via a sliding windowed update. Then, with the estimation of μ_d and σ_d^2, we can calculate the required prediction length for an error bound ϵ as:

$$\hat{l}_{d,\mathcal{B}} = \mu_d + \frac{\sigma_d}{\sqrt{|\mathcal{B}|}} \cdot \Phi^{-1}(1-\epsilon),$$

where Φ^{-1} is the inverse of the standard normal cumulative distribution function. This formula gives us a prediction bound that will be exceeded only with probability ϵ, allowing precise control over the trade-off between resource efficiency and eviction risk. We denote the virtual batch corresponding to the sliding window for request r_i as $\mathcal{B}_{\mathcal{W}_i}$.

Similarly, this method applies to input length prediction, as we make no distributional assumptions beyond the applicability of the Central Limit Theorem. In fact, this method applies to mixed scenarios, demonstrating its ease of use and generalizability. We provide experimental validation in the supplementary Appendix A. Additionally, we introduce simple correction strategies for handling long-tail distributions to further stabilize prediction accuracy in skewed workloads.

Further, we translate the batched length estimation into computation resource estimation, such as FLOPs(F) and memory(M).

$$\begin{aligned} F_{\text{prefill}} &= \alpha \sum_{r_i \in \mathcal{Q}_{\text{new}}} l_{p,i}^2 + \beta \sum_{r_i \in \mathcal{Q}_{\text{new}}} l_{p,i} + \gamma, \\ F_{\text{decode}} &= \lambda \sum_{r_i \in \mathcal{Q}_{\text{active}}} (l_{p,i} + l_{d,i}) \cdot \hat{l}_{d,\mathcal{B}} + \mu, \\ M_{\text{prefill}} &= c_{\text{kv}} \cdot \sum_{r_i \in \mathcal{Q}_{\text{new}}} l_{p,i} \\ M_{\text{decode}} &= c_{\text{kv}} \cdot \sum_{r_i \in \mathcal{Q}_{\text{active}}} (l_{p,i} + l_{d,i}) \end{aligned} \tag{2}$$

where c_{kv} is the size of one token, and α, β, γ, λ, and μ are coefficients that can be calibrated offline. The details are listed at Appendix B. Our prediction-based method utilizes these four metrics to estimate the required number of GPUs.

4.2 Intra-GPU Computation Scheduling with accurate estimation

Previous request batching methods estimate memory requirements at individual request granularity, with simplistic approaches to predicting future needs. For example, an eviction-free strategy, with estimation $e(l_{p,i}, l_{d,i}) = l_{m,i} - l_{d,i}$, reserves the maximum possible remaining space for each request [9]. On the other hand, an aggressive strategy, with $e(l_{p,i}, l_{d,i}) = 1$, assumes minimal future generation [11]. Both approaches lead to suboptimal resource utilization: they either waste resources via over-allocation or risk frequent evictions when actual lengths exceed predictions.

Our statistical prediction performs best when applied to requests in a sliding window rather than individual requests. We redesign the inter-GPU scheduling algorithm to leverage this batch-level prediction capability, shifting from per-request memory reservation to batch-level memory management:

$$c(\mathcal{B}) := \underbrace{c(\mathcal{B})}_{\text{last usage}} + \underbrace{\sum_{r_i \in \mathcal{Q}_{\text{new}}} c_{\text{kv}} \cdot (2 \cdot \mathbb{1}_i - 1) \hat{l}_{d,\mathcal{B}_{\mathcal{W}_i}}}_{\text{estimation update}}, \tag{3}$$

where $\mathbb{1}_i$ indicates request r_i is enqueued(1) or poped(0). The second term is updated to reflect the estimated lengths of newly enqueued requests and the KV memory released by finished requests as they depart.

This contrasts with traditional approaches, where future memory is estimated per request:

$$\tilde{c}(\mathcal{B}) = \sum_{r_i \in \mathcal{B}} c_{\text{kv}} \cdot (\underbrace{l_{p,i} + l_{d,i}}_{\text{current usage}} + \underbrace{e(l_{p,i}, l_{d,i})}_{\text{predicted needs}}). \tag{4}$$

Compared to the existing formulation, the batch-aware one has two key features:
(1) Statistical Efficiency: By aggregating predictions over batches, we benefit from reduced variance in our estimates as shown in Section 4.1. While individual request predictions might have high error rates, batch-level predictions have much tighter error bounds.
(2) Memory Sharing: Our approach requires memory sharing among requests within a batch to apply the batch-level prediction. This enables better flexibility so that some requests can use less than their predicted allocation while others use more, as long as the batch total remains within limits.

4.3 Inter-GPU Resource Management and Request Dispatching

Efficiently allocating computational resources while maintaining service quality requires both accurate workload estimation and robust dispatch mechanisms. Our approach directly estimates computational requirements (FLOPs) from predicted workload characteristics rather than relying on generic utilization metrics.

Our system operates with two parallel time windows: a longer window for GPU instance management and a shorter window for request dispatching. For each GPU management window, we use estimated computational requirements for both phases, as shown in Equation 2.

Based on these estimates and known GPU computational capacity, we determine the minimum instances needed to satisfy target latency requirements in the management window:

$$\begin{aligned} n_p &= \lceil F_{\text{prefill}}/(C_p \cdot t_{\text{target,p}}) \rceil, \\ n_d &= \lceil F_{\text{decode}}/(C_d \cdot t_{\text{target,d}}) \rceil, \end{aligned} \tag{5}$$

where C_p and C_d are the computational capacity of prefill and decode instances, and $t_{\text{target,p}}$ and $t_{\text{target,d}}$ are the target latencies for prefill and decode phases, respectively. Memory constraints and instance limits are omitted for simplicity. The final GPU allocation takes the maximum of the GPU instance counts needed to satisfy both compute and memory constraints.

To ensure robustness against prediction errors and sudden load spikes, we propose two complementary algorithms for the dispatch window: Dynamic Resource Scheduling (Algorithm 1, taking prefill for instance) for regular request dispatching and Spike Reaction (Algorithm 2) for handling unexpected load spikes. Spikes occur only for outliers, which are of low frequency according to Table 1. It is worth noting that the algorithms presented here are for scheduling prefill instances; decode instance scheduling adheres to the same design principles.

Algorithm 1 Dynamic Resource Scheduling Algorithm

Require: Request queue $\mathcal{Q}$, idle instances $\mathcal{I}_{\text{idle}}(t)$, running instance set $\mathcal{I}_{\text{active}}(t)$, deadline d_i for request r_i
Ensure: Task dispatch results and spike reaction signals

```
1:  Q_spike ← ∅
2:  while Q ≠ ∅ do
3:      Extract highest priority request r_i from Q
4:      F_i ← α · l_{p,i}^2 + β · l_{p,i} + γ
5:      if I_idle(t) ≠ ∅ then
6:          Get G ∈ I_idle(t) and update I_idle(t), I_active(t)
7:      else
8:          Select G ∈ I_active(t) with earliest completion
9:          Update the SLO acceptable time t_s based on d_i
    and estimate execution time t_e if r_i is enqueued
10:         If SLO can be met, dispatch r_i to G;
11:         Otherwise put to Q_spike
12:     end if
13: end while
14: if Q_spike ≠ ∅ then
15:     Call Spike Reaction(Q_spike)
16: end if
```

The Dynamic Resource Scheduling algorithm operates in a shorter time window, continuously dispatching requests to available instances. It maintains a global task queue $\mathcal{Q}$ and tracks both idle instances $\mathcal{I}_{\text{idle}}(t)$ and running instances $\mathcal{I}_{\text{active}}(t)$. For each request, it calculates compute requirements based on input length (line 4) and attempts to place it optimally.

If idle instances are available, the request is immediately assigned (line 6). Otherwise, the algorithm selects the running instance with the earliest expected completion time (line 8) and evaluates whether adding the new request would violate deadline constraints (lines 9-11). This approach maximizes instance utilization while respecting service quality constraints.

When the algorithm determines that existing instances cannot handle incoming requests within their deadlines (line 15), it triggers the Spike Reaction mechanism (Algorithm 2). It rapidly activates new instances to handle sudden workload increases that exceed planned capacity. Operating independently of the regular management cycle, it forms the largest possible batches of pending requests to efficiently utilize new resources.

Algorithm 2 Spike Reaction Resource Allocation Algorithm

Require: Request queue $\mathcal{Q}_{\text{spike}} = \{r_1, r_2, \ldots, r_n\}$ (sorted by deadline), computational capacity C_p, current time t, minimum viable utilization $U_{\text{threshold}}$
Ensure: Allocation results and instance activation decisions

```
1:  while Q_spike ≠ ∅ do
2:      for b ← 1 to n do            ▷ Try different batch sizes
3:          If current instance memory is insufficient, break
4:          Compute execution time t_e by Equation 2.
5:          if t_e breaks deadline then
6:              Pop b − 1 requests as a batch B_spike.
7:              break
8:          end if
9:      end for
10:     Calculate the actual utilization U_actual
11:     if U_actual ≥ U_threshold and new instances are avialiable
    then
12:         Activate new instance and dispatch B_spike
13:     else
14:         mark the B_spike as "cannot satisfy"
15:     end if
16: end while
```

Spike Reaction evaluates different virtual batch sizes to maximize utilization (lines 2-9). For each potential configuration, it selects the largest batch that can meet deadline requirements (lines 5-8). A key feature of this algorithm is its utilization-aware activation policy (line 10). It only activates new instances when predicted utilization exceeds the threshold $U_{\text{threshold}}$, preventing resource wastage on small spikes that do not harm the system's service quality. In such cases, for unmet requests arising from these spikes that do

not harm the system's service quality, they are queued, temporarily foregoing SLO guarantees, thus ensuring that even reactive scaling remains economically efficient.

These two algorithms operate in concert: the Dynamic Resource Scheduling continuously optimizes request placement during normal operation, while the Spike Reaction mechanism provides a safety net that rapidly responds to unexpected load changes between regular planning cycles. Together, they ensure both resource efficiency during steady-state operation and robustness during load spikes.

5 Implementation

To demonstrate the effectiveness of our proposed algorithms, we implement PiLLM by extending LightLLM [12]. We chose LightLLM as our foundation because its token-level KV cache management aligns well with our requirements for intra-GPU batch-level memory sharing and disaggregated prefill/decode paradigm. In this section, we describe how the key algorithms function in our real system implementation.

5.1 System Architecture and Request Flow

PiLLM follows a hierarchical architecture designed to efficiently manage LLM inference workloads. As shown in Figure 3, our system consists of three main layers: (1) the API layer serving as the interface to client applications, (2) the global scheduling layer handling resource management and request dispatching, and (3) the execution instances where actual inference takes place.

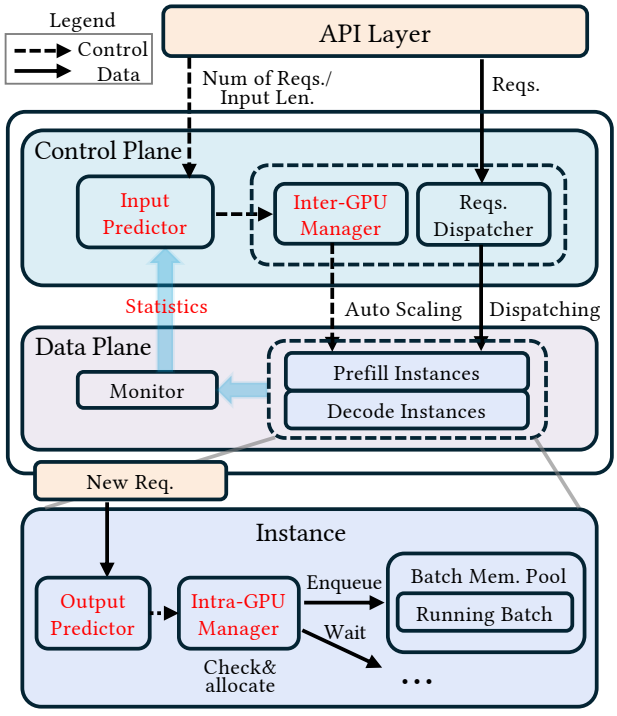

Figure 3. PiLLM System Architecture

At the global scheduling layer, we implement two key components: a dispatcher that routes incoming requests based on their computational characteristics, and an instance count manager that governs GPU resource allocation. These components collaboratively implement our dynamic resource scheduling algorithm (Algorithm 1). When traffic spikes occur, our spike reaction algorithm (Algorithm 2) rapidly activates additional resources to maintain service quality.

The execution layer consists of multiple inference instances, each potentially spanning multiple GPUs. We deploy two types of specialized instances: prefill instances that handle input token processing, and decode instances for generating output tokens. Each instance incorporates a batching scheduler that applies our predictive length-aware policies to maximize throughput while meeting latency constraints. Additionally, instances provide a fast resume mechanism enabling rapid response to traffic spikes, via a multi-level parameter cache like ServerlessLLM [3].

From the perspective of user requests, the processing follows a defined flow through this architecture:
(1) User requests enter through the API layer;
(2) The global dispatcher analyzes request characteristics and determines routing based on Algorithm 1;
(3) Requests are assigned to either active prefill instances or queued for spike reaction;
(4) Prefill instances process initial tokens and transfer computational state to decode instances after each layer;
(5) Decode instances continue generation while prefill resources become available for new requests;
(6) Completed responses return via the API layer to the user.

This architecture allows PiLLM to effectively separate control plane (scheduling and resource management) from data plane (model execution), , where the control plane makes sure the instances meet SLO constraints, and the data plane tries to fully utilize the allocated resource. By specializing instances for distinct computational patterns, our system achieves both high resource utilization and responsive adaptation to changing workloads, enabling a balance between service quality and efficiency.

5.2 Statistical Data Collection for Workload Resource Prediction

PiLLM implements a multi-level statistical data collection framework to power its predictive scheduling mechanisms while minimizing system overhead.

For inter-GPU scheduling, we collect data at two strategic points in the request lifecycle. For prefill resource prediction, we leverage input length information already available at the API layer, where requests first enter the system. This approach incurs virtually zero overhead as we simply capture data that exists in the request processing pipeline. For decode resource prediction, we collect output length statistics directly from decode instances. To avoid communication overhead, we aggregate these statistics at configurable time

intervals rather than after every token generation. This periodic sampling provides sufficient data accuracy for our prediction models while minimizing system impact.

For intra-GPU scheduling, our prediction framework operates at the batch level, primarily using current output lengths of active requests—information already maintained by the batch scheduler as part of normal execution state tracking. We predict the required memory with the predicted left generation in batch granularity.

Our statistical models primarily track and utilize average values and standard deviations of a sliding window, which provide sufficient information to apply our prediction algorithms while being computationally lightweight during runtime.

5.3 Memory Budget Sharing for Batch-Aware Scheduling

Our system leverages token-level KV cache management inherited from LightLLM [12], which we extend to implement our batch-aware memory sharing approach. Rather than allocating fixed memory blocks to each request, our implementation structures the KV cache as a linked list of token slots, enabling flexible memory (KV cache) allocation and deallocation. When any request requires additional space for new tokens, the system can allocate from the shared pool with minimal overhead.

5.4 Prefill-Decode Disaggregation For Fine-grained GPU Reduction

We leverage prefill-decode disaggregation to achieve fine-grained GPU reduction based on our predictive scheduling algorithms. While disaggregation is typically used for better QoS, we recognize its potential for significant resource savings by separating phases with different computational characteristics: prefill requires high computational throughput for parallel processing of input tokens, while decode is less computationally intensive and more memory-bound, enabling much higher GPU savings.

To maximize efficiency, our system maintains separate pools of prefill and decode instances, each optimized for its specific workloads. When allocating resources for a new request, our dispatcher pre-assigns both prefill and decode instances simultaneously, establishing a guaranteed execution path before computation begins. This approach minimizes transition costs, as the KV cache state can be transferred progressively after each attention layer's computation rather than waiting for the entire prefill phase to complete.

6 Experiments

6.1 Experimental Setup

Our evaluation of PiLLM focuses on its ability to achieve resource efficiency while maintaining service quality across diverse workloads. Here, we describe the experimental hardware/software platform, datasets, metrics, and setup comparison baselines.

Hardware. We conduct our experiments on 8 NVIDIA H800 GPUs with NVLink all-to-all interconnect, 2 Intel Xeon 6448Y CPUs, and 1TB of DDR memory.

Software. Our software stack consists of PyTorch 2.1, CUDA 12.1, and custom extensions to the LightLLM framework. We implement both our system and baselines with the same underlying GPU kernel implementations to ensure fair comparisons of inter- and intra-GPU scheduling strategies.

Datasets. We use both public datasets and synthetic workloads derived from production patterns:
BurstGPT [16]: A public dataset of ChatGPT tracing, containing anonymized request lengths with timestamps.
MoonCake [14]: Released by Kimi AI, with heterogeneous length patterns and long-tail distributions.
Production-derived workloads: We created additional datasets containing only length information and timing metadata (with all content and personally identifiable information removed). *Conversation* contains multi-turn chat histories with varied context lengths. *Document* traces long-context workloads with extensive input texts. *Assistant* contains task-oriented interactions with moderate input/output lengths

These datasets collectively cover diverse request patterns, with average input lengths ranging from 32 to over 120,000 tokens and output lengths from 30 to 25,000 tokens, which is shown in Table 1. All exhibit pronounced long-tail distributions, with the 99th percentile typically 3-10 times larger than the 90th percentile—creating significant resource allocation challenges that PiLLM specifically addresses.

Metrics. We focus on two categories of metrics directly relevant to LLM inference providers and users: resource efficiency and service quality metrics.

For resource efficiency, PiLLM is evaluated using two primary types of measurements. For the inter-GPU scheduler, the GPU Saving Factor quantifies the ratio of GPUs required by baseline systems compared to those required by PiLLM under equivalent workloads and quality targets. This metric directly reflects the cost reduction potential for deployment. For the intra-GPU scheduling, memory Utilization captures the percentage of intra-GPU memory actively used for KV cache during operation, indicating how efficiently each allocated GPU is being used.

For service quality, our primary metric is the SLO Satisfaction Rate, which represents the percentage of requests meeting their target latency requirements. This metric most directly reflects the user experience while providing a clear measure of system reliability. To establish meaningful SLOs, we take an individualized approach tailored to each specific request rather than using fixed deadlines across all requests.

Table 1. Key Characteristics of Experimental Datasets

Dataset	Input Length					
Percentile	90%		99%		Overall	
	Mean	Std	Mean	Std	Mean	Std
BurstGPT	1792	367	3395	719	11099	781
Mooncake	16858	3198	64266	7757	125546	11121
Assistant	738	137	1604	259	2398	304
Conversation	485	77	3093	394	16670	710
Document	39549	4973	120841	18940	123831	22437
Dataset	**Output Length**					
Percentile	90%		99%		Overall	
	Mean	Std	Mean	Std	Mean	Std
BurstGPT	271	69	1494	183	3206	272
Mooncake	509	163	910	212	2000	244
Assistant	339	88	593	116	1024	129
Conversation	1227	377	1796	426	25589	521
Document	419	70	599	89	736	97

Specifically, we first execute each request in our test datasets on the baseline system with maximum resource allocation to measure its reference performance. A single fixed SLO deadline would be inappropriate: if calibrated for long requests, it would be too permissive for shorter interactions, masking potential performance degradation in common use cases; if calibrated for short requests, it would create impossible targets for inherently longer operations. We then set that request's SLO target for PiLLM as 20% higher than its baseline execution time. This approach is necessary due to the extreme variation in request characteristics across our workloads, from simple assistant interactions requiring only a few hundred tokens to complex document processing tasks with over 100,000 tokens. Notably, the actual resource savings substantially exceed 20%, as shown in Table 3.

In our ablation studies, we additionally examine P99 latency (the 99th percentile response time) of TTFT(Time-to-first-token) and TPOT(Time-per-output-token) to better understand tail performance characteristics, which are particularly important for production systems where consistent service quality must be maintained even at the extremes of the distribution.

These complementary metrics allow us to comprehensively evaluate PiLLM's ability to balance the fundamental trade-off between resource efficiency and service quality that defines effective LLM inference systems.

Baselines. We compare PiLLM against two levels:
Inter-GPU management: We compare PiLLM's dynamic resource allocation with: (a) Metrics-based scaling that adjusts GPU count based on utilization metrics, which is used for service quality comparison. (b) Fixed maximum resource allocation, representing the conservative approach commonly used in production.
Intra-GPU batching: We compare PiLLM's batching strategy with leading systems. (a) vLLM's greedy approach that maximizes immediate resource utilization [11]. (b) PastFuture's conservative strategy that prioritizes non-eviction of requests based on probabilistic sampling [4]. (c) SGLang's configurable rate-based approach that attempts to balance utilization and stability [20]. For all baseline systems, we use their default parameter settings as specified in their original implementations to ensure fair comparison.

For fair comparison, all systems run the same underlying model (LLaMA-3.1 8B [5]) and serve identical workloads. In PiLLM configurations, we limit scaling between 1-4 GPU instances per phase, establishing a theoretical maximum saving factor of 4 times. Also, for the intra-GPU batching, we make sure the requests are in the same order, in case of unstable performance.

Table 2. GPU Savings and SLO Satisfaction Rate Across Different Workloads

Dataset	Avg. GPU Saving	SLO Satisfaction	
		Prefill	Decode
BurstGPT	2.01×	97.9%	100%
MoonCake	2.02×	100%	100%
Assistant	1.62×	99.1%	100%
Conversation	2.56×	98.6%	100%
Document	3.06×	100%	100%

6.2 Overall System Performance

The primary goal of PiLLM is to reduce GPU resource requirements while maintaining acceptable service quality. Table 2 presents a comprehensive view of our system's performance across different workloads, comparing GPU savings and SLO satisfaction rates against the baseline systems.

PiLLM achieves substantial GPU savings across all tested scenarios, with reduction factors ranging from 1.62 times for assistant workloads to 3.06 times for document processing tasks. These savings translate directly to reduced operational costs or increased capacity at the same cost. The greatest savings occur in document processing workloads due to their highly variable input lengths, where static allocation approaches waste significant resources.

Importantly, our SLO satisfaction rates remain consistently high, with over 97.9% satisfaction for prefill operations and 100% for decode operations across all workloads. This demonstrates PiLLM's ability to maintain service quality while dramatically reducing resource needs, validating our statistical approach to resource allocation.

The minor SLO degradation observed in the prefill stage is an expected consequence of our dynamic resource allocation strategy. PiLLM deliberately reduces the number of active GPUs while improving their utilization through increased batch sizes. For the computation-intensive prefill stage, these larger batches inevitably extend processing time for some requests, occasionally exceeding tight SLO thresholds. In contrast, the decode stage maintains perfect SLO satisfaction despite batching because decode operations are relatively shorter in duration, making them less susceptible to SLO violations even with the additional latency introduced by larger batch sizes. This controlled trade-off between resource efficiency and processing time represents an effective balance—achieving substantial GPU savings with only a negligible impact on user experience.

6.3 Inter-GPU Saving Analysis

A key innovation in PiLLM is its phase-specific resource allocation strategy. Table 3 breaks down GPU savings by inference phase, revealing that our approach achieves substantially different resource efficiency between phases.

Across all workloads, decode phase savings are remarkably consistent, ranging from 3.72 times to 4.00 times, approaching the maximum saving of 4 times permitted by our experimental configuration (which allocated a minimum of 1 instance out of 4 available per phase). This consistency reflects the relatively low computation density of token generation during decoding. In contrast, prefill phase savings exhibit higher variance, ranging from nearly neutral (1.03 times) for assistant workloads to substantial (2.48 times) for document processing. This variation reflects the diverse computational demands of varied prompt lengths during prefill.

Table 3. Per-Phase GPU Saving Breakdown

Phase	Average	Prefill	Decode
BurstGPT	2.01×	1.35×	3.85×
MoonCake	2.02×	1.39×	3.72×
Assistant	1.62×	1.03×	3.74×
Conversation	2.56×	1.88×	4.00×
Document	3.06×	2.48×	3.99×

This asymmetry in savings validates our prefill/decode disaggregated approach. Traditional systems that use the same GPU allocation for both phases are constrained by the most resource-intensive phase requirements, leading to significant underutilization during lighter phases. By allocating resources independently, PiLLM can assign just enough GPUs to each phase, avoiding the inefficiency of one-size-fits-all approaches.

Figure 4 demonstrates PiLLM's superior adaptation to workload variations. The top graph shows the incoming request load over time with two significant spikes around window indices 10 and 55. The bottom graph compares our Spike Reaction (SR) approach with traditional Metrics Reaction (MR). When load spikes occur, SR immediately scales up GPU instances to handle the increased demand, while MR exhibits a characteristic delay of one full scheduling window before responding. This delayed reaction in MR directly impacts SLO compliance during critical periods of high demand, as resources remain insufficient during the adaptation lag. While MR may achieve marginally better resource utilization in steady states, this comes at the cost of compromised service quality during transitions.

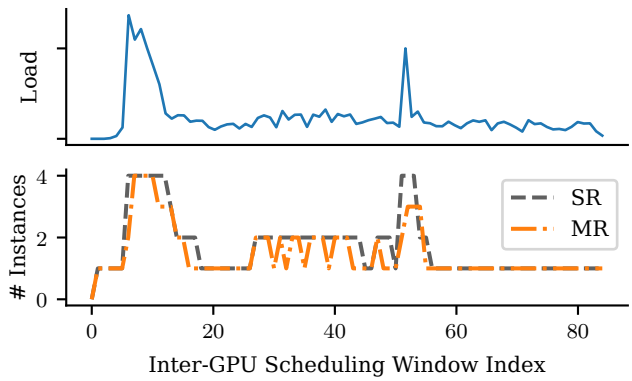

Figure 4. Auto-scaling comparison with hardware metrics controlled methods. SR: Spike Reaction; MR: Metrics Reaction. MR shows lagging response to load changes.

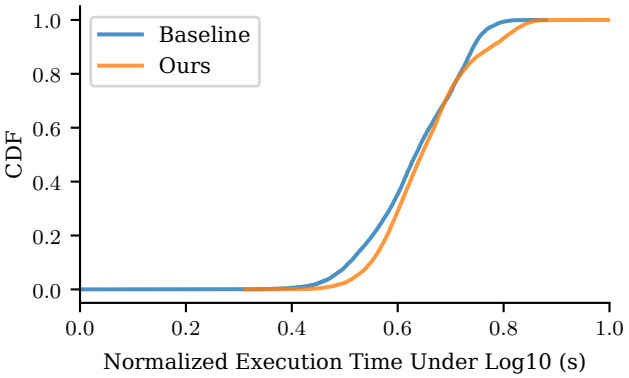

Figure 5. CDF Of Execution Time

The cumulative distribution function in Figure 5 plots the distribution of execution times (shown on a logarithmic scale for better visualization across different time ranges) for both baseline and PiLLM approaches. The slight rightward shift in PiLLM's curve indicates marginally increased execution times compared to the fixed allocation baseline. This shift represents the classical throughput-latency tradeoff: by increasing batch sizes and improving GPU utilization, PiLLM

achieves higher throughput at the cost of slightly longer per-request processing times. Importantly, both curves exhibit similar shapes and convergence in the tail region, demonstrating that PiLLM's optimization preserves SLO guarantees for most users while significantly reducing resource requirements. The logarithmic scale highlights that this performance difference remains within acceptable bounds across all percentiles of the distribution.

6.4 Intra-GPU Efficiency

Beyond inter-GPU resource allocation, PiLLM also optimizes memory utilization within each GPU through its length-aware batch scheduling. Table 4 compares PiLLM's intra-GPU performance metrics against vLLM, SGLang, and PastFuture across different workloads, showing significant improvements in batch concurrency and memory utilization while maintaining low request eviction rates.

PiLLM achieves memory utilization rates between 78.9% and 96.1% across different workloads, significantly higher than the 45-70% typical in production systems using fixed-size KV cache allocation. This improved utilization stems from our statistical prediction approach, which allows for safe overcommitment of GPU memory. By predicting batch-level memory requirements rather than worst-case individual needs, PiLLM can accommodate more concurrent requests while maintaining minimal service disruption.

The results in Table 4 underscore interesting performance trade-offs among different batching approaches. While vLLM achieves slightly higher memory utilization, reaching within 0.3% of PiLLM across various workloads, it also suffers from significantly higher eviction rates, peaking at 68.39% for *Conversation* workloads. In contrast, conservative approaches like PastFuture and SGLang maintain zero evictions but substantially underutilize available resources. Both our approach and PastFuture leverage statistical information for better resource management. However, PastFuture performs sampling for every request, potentially overlooking the low-variance nature that statistical insights can offer. PiLLM, on the other hand, finds a harmonious balance, achieving near the utilization efficiency of aggressive schedulers while keeping eviction rates low.

6.5 Ablation Studies

To understand how individual components contribute to system performance, we conducted ablation studies on both inter-GPU and intra-GPU optimizations.

For inter-GPU resource management, Table 5 demonstrates the impact of our spike reaction mechanism on system responsiveness during sudden traffic increases. The table shows the percentage increase in Time-To-First-Token (TTFT) tail latency during traffic spikes compared to normal operations. Across all workloads, our spike reaction mechanism keeps latency increases significantly lower than metric-based reactions, with particularly dramatic improvements for Document workloads (102% vs 491%) and MoonCake workloads (116% vs 353%). This confirms that proactive resource allocation based on statistical prediction substantially outperforms reactive scaling approaches during critical traffic fluctuations.

For intra-GPU scheduling, Figure 6 illustrates how prediction accuracy affects memory utilization and request eviction rates across two representative workloads. The horizontal axes represent the increasing percentile accuracy of memory usage prediction for individual requests, from the 25th to the

Table 4. Intra-GPU Performance Comparison Across Scheduling Algorithms

Dataset	Batching Strategy	Eviction Rate	Avg. Batch Size	Mem. Util.
BurstGPT	Ours	0.01%	282.21	78.93%
	PastFuture	0.00%	223.16	62.42%
	SGLang	0.00%	231.21	64.67%
	vLLM	10.38%	283.12	79.18%
MoonCake	Ours	0.07%	37.28	96.05%
	PastFuture	0.00%	36.66	94.46%
	SGLang	0.00%	36.24	93.37%
	vLLM	1.71%	37.33	96.18%
Assistant	Ours	0.32%	798.33	82.03%
	PastFuture	0.00%	645.5	66.33%
	SGLang	0.00%	483.30	49.66%
	vLLM	6.83%	801.13	82.31%
Conversation	Ours	0.05%	639.43	91.06%
	PastFuture	0.00%	365.92	52.15%
	SGLang	0.00%	431.68	61.53%
	vLLM	68.39%	654.04	91.50%
Document	Ours	0.53%	43.37	93.78%
	PastFuture	0.00%	42.80	92.58%
	SGLang	0.00%	41.93	90.70%
	vLLM	2.85%	43.42	93.88%

Table 5. Spike Reaction Analysis On Tail Latency of TTFT

Dataset	Reaction Type	
	Spike	Metric
BurstGPT	139%	223%
MoonCake	116%	353%
Assistant	166%	159%
Conversation	118%	130%
Document	102%	491%

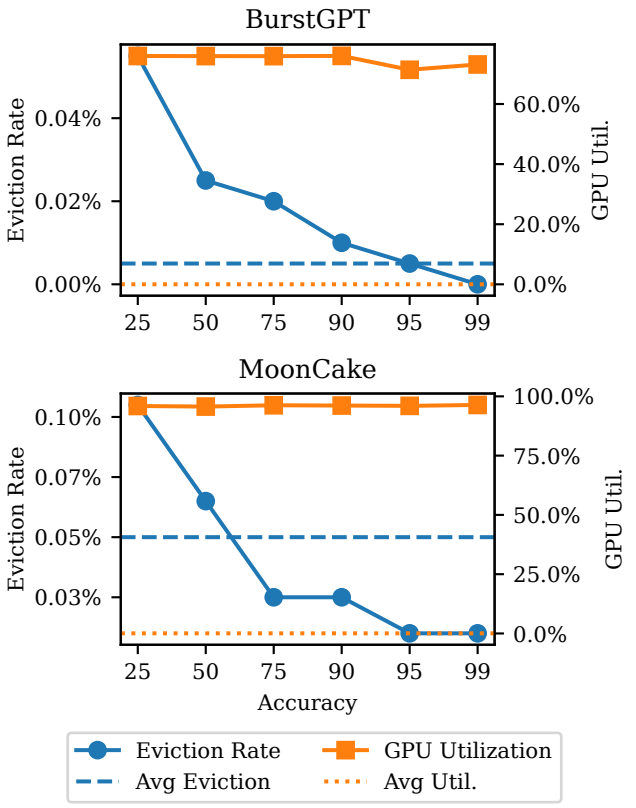

Figure 6. Impact of Prediction Accuracy on Intra-GPU Utilization and Eviction Rates

99th percentile. Importantly, while these accuracy levels refer to per-request predictions, PiLLM performs scheduling at batch granularity, which provides inherent resilience against individual prediction errors. This batch-level approach explains why even moderate prediction accuracies (25-50%) yield strong performance results—the statistical aggregation across multiple requests in a batch naturally compensates for individual prediction variances.

To ensure the intra-GPU estimation of our scheduling system, we implemented techniques to stabilize the conversion from length to execution time. We employed AOT(Ahead-of-time) compilation to avoid runtime overhead and CUDA graph execution to reduce jitters from memory allocation. It used a series of kernels with fixed-sized intermediate workspaces that leverage pre-compiled operations and pre-allocated memory. These optimizations significantly improved time prediction accuracy, limiting error to under 20% for decode phases and under 10% for prefill operations, which ensures precise scheduling in PiLLM. Due to space constraints, detailed implementation and characterization are provided in the Appendix B.

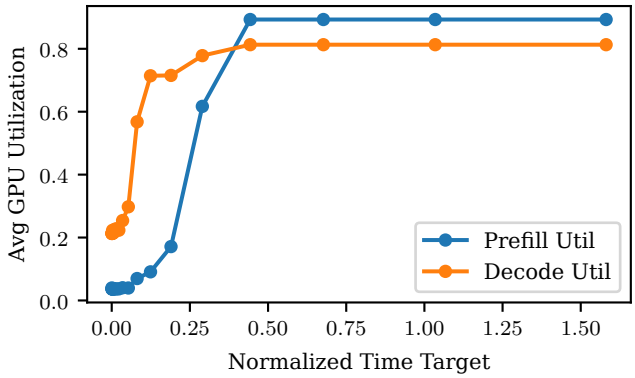

Figure 7. Impact of Execution Time Goal on the GPU Utilization.

To extend our validation beyond available hardware, we simulated large-scale production deployments. Figure 7 illustrates the fundamental trade-off between time targets and GPU utilization. The simulation demonstrates that relaxing these time targets can substantially improve system-wide utilization, although utilization eventually saturates. Additionally, decode operations (orange) consistently achieve higher efficiency than prefill operations (blue) due to their more predictable execution patterns, confirming our experimental findings at scale.

7 Conclusion

PiLLM demonstrates that statistical batch-level prediction can effectively address the efficiency challenges in LLM inference systems. By dynamically allocating resources at both inter-GPU and intra-GPU levels, our approach reduces GPU requirements by 1.6-3.1× while maintaining over 97.9% SLO compliance. While our method excels at handling workload spikes, we observe varying gains across different workload types, with a controlled trade-off between resource efficiency and per-request latency.

Our current implementation has limitations in centralized request queueing optimization and may face challenges when scaling to extremely large clusters with thousands of GPUs, due to the centralized controller. Besides, it should be practical that prefill and decode share the same instance pool, as they use the same copy of parameters. This might enable prefill-decode auto-converting, which may benefit according to the fact that decode keeps a relatively low number of active instances while prefill utilizes almost all resources.

Acknowledgement

This work was supported by the Postdoctoral Fellowship Program and China Postdoctoral Science Foundation (No. BX20250487).

References

[1] Aliyun. [n. d.]. *Deploy LLMs as elastic inference services in ACK Edge clusters in hybrid cloud environments.* https://www.alibabacloud.com/help/en/ack/ack-edge/use-cases/deploy-llm-inference-service-in-hybrid-cloud-scenarios

[2] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, and Junxiao Song. 2025. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* doi:10.48550/arXiv.2501.12948

[3] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. *ServerlessLLM: Locality-Enhanced Serverless Inference for Large Language Models.* http://arxiv.org/abs/2401.14351

[4] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, and Zaijun Wang. 2025. Past-Future Scheduler for LLM Serving under SLA Guarantees. In *ASPLOS '25: 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Rotterdam Netherlands). ACM, 798–813. doi:10.1145/3676641.3716011

[5] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, and Abhishek Kadian. 2024. *The Llama 3 Herd of Models.* doi:10.48550/arXiv.2407.21783

[6] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. 2020. Serving {DNNs} like Clockwork: Performance Predictability from the Bottom Up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20).* 443–462. https://www.usenix.org/conference/osdi20/presentation/gujarati

[7] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, and Shichao Pei. 2024. *Large Language Model Based Multi-Agents: A Survey of Progress and Challenges.* doi:10.48550/arXiv.2402.01680

[8] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, and Jeff Rasley. 2024. *DeepSpeed-FastGen: High-throughput Text Generation for LLMs via MII and DeepSpeed-Inference.* doi:10.48550/arXiv.2401.08671

[9] HuggingFace. [n. d.]. *Text Generation Inference.* https://huggingface.co/docs/text-generation-inference/index

[10] KServe. [n. d.]. *Home - KServe Documentation Website.* https://kserve.github.io/website/latest/

[11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, and Lianmin Zheng. 2023. *Efficient Memory Management for Large Language Model Serving with PagedAttention.* http://arxiv.org/abs/2309.06180

[12] ModelTC. 2025. *ModelTC/Lightllm.* ModelTC. https://github.com/ModelTC/lightllm

[13] OpenAI. [n. d.]. *Introducing Deep Research.* https://openai.com/index/introducing-deep-research/

[14] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. *Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving.* doi:10.48550/arXiv.2407.00079

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, and Llion Jones. 2023. *Attention Is All You Need.* doi:10.48550/arXiv.1706.03762

[16] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, and Zhenheng Tang. 2024. *BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems.* doi:10.48550/arXiv.2401.17644

[17] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, and Brian Ichter. 2023. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* doi:10.48550/arXiv.2201.11903

[18] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for {Transformer-Based} Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22).* 521–538. https://www.usenix.org/conference/osdi22/presentation/yu

[19] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. 2023. {SHEPHERD}: Serving {DNNs} in the Wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23).* 787–808. https://www.usenix.org/conference/nsdi23/presentation/zhang-hong

[20] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, and Jeff Huang. 2024. *SGLang: Efficient Execution of Structured Language Model Programs.* doi:10.48550/arXiv.2312.07104

[21] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. 2023. *Response Length Perception and Sequence Scheduling: An LLM-Empowered LLM Inference Pipeline.* http://arxiv.org/abs/2305.13144

[22] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, and Yibo Zhu. 2024. *DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving.* http://arxiv.org/abs/2401.09670

[23] Botao Zhu, Chen Chen, Xiaoyi Fan, and Yifei Zhu. 2025. *LLMSched: Uncertainty-Aware Workload Scheduling for Compound LLM Applications.* doi:10.48550/arXiv.2504.03444

A Statistical Prediction

According to Table 1, the tail 10% of the distribution can alter the average by tens of times, which means the Equation 1 might be unstable as it relies on the observed mean of the distribution. One direct solution is to omit the long tail part:

$$\hat{l}_{d,\mathcal{B},k\%} = \mu_{d,k\%} + \frac{\sigma_{d,k\%}}{\sqrt{|\mathcal{B}|}} \cdot \Phi^{-1}(1-\epsilon), \tag{6}$$

where the mean and standard deviation are replaced by those of the first $k\%$ lengths of $\mathcal{B}$ in ascending order. The only dependency between our methods and the distribution is the mean and standard deviation. Therefore, inherently, there is no limitation to mixed scenarios. To verify this, we conducted experiments across various mixed workloads as shown in Table 6.

Table 6. Intra-GPU scheduling performance in mixed workload scenarios

Dataset	Eviction Rate	Average Batch Size	Memory Utilization
MoonCake	0.01%	282.21	78.93%
BurstGPT	0.07%	37.28	96.05%
Moon.+Burst.	0.48%	143.13	93.3%
Document	0.53%	43.37	93.78%
Conversation	0.05%	639.43	91.06%
Assistant	0.32%	798.33	82.03%
Doc.+Conv.+Asst.	0.44%	58.00	95.48%

Our experimental results demonstrate that our statistical prediction approach maintains high performance even in mixed workload scenarios. The mixed workloads achieve comparable or even superior memory utilization compared to single-scenario workloads while maintaining low request pause rates. This confirms that our method generalizes well to complex, real-world deployment scenarios where request patterns may come from multiple sources simultaneously.

B Predictable Execution Time

Execution time uncertainties also pose challenges for effective inter-GPU scheduling. Without addressing these , reliable inter-GPU scheduling becomes infeasible. We identified two primary sources:

1. Dynamic memory allocation and deallocation operations.
2. Just-in-time (JIT) kernel compilation triggered by variable input/output lengths;

Most LLM inference frameworks already pre-allocate KV cache, while the intermediate workspace memory is still dynamically managed as the requirements are related to the sum of sequence lengths.

We improve execution time predictability by redesigning kernel memory layouts using a chunk-based approach:

1. Every request is split into fixed-size chunks.
2. The kernel allocates a fixed number of chunk workspaces in GPU memory.
3. The system breaks requests into chunks to fill available workspace slots.

This approach makes memory allocation independent of the number of requests, with the only cost being the pre-allocation of chunks (some of which may remain empty). Importantly, this design accommodates varying input/output lengths and batch sizes while maintaining consistent memory usage patterns, which allows for CUDA Graph optimization. It eliminates Triton kernel recompilation time, as chunk sizes remain constant

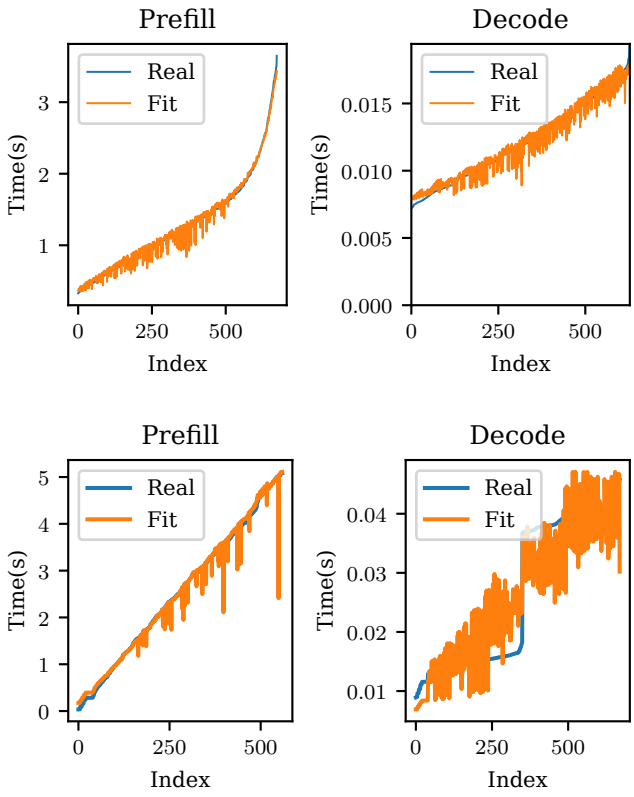

Figure 8. Prefill And Decode Execution Time Fit for LLaMA 3.1 8B and DeepSeek V2 Lite

After applying the techniques described above, our execution time prediction accuracy improved significantly. Figure 8 shows the fitting results for LLaMA-1.8B and DeepSeek V2 Lite models respectively, demonstrating acceptable prediction quality. In these figures, the x-axis represents the sorted index of execution samples (increasing order of real execution time), while the y-axis shows the time. Although the fit errors remain, the magnitude is substantially reduced to about 20%. The improved prediction accuracy enables reliable estimation of execution time for both prefill and decode phases across different models, which ensures Flops-to-GPU-count conversion is possible.